

DEVOPS HOME LAB HANDBOOK

BUILD • AUTOMATE • DEPLOY • MONITOR • TROUBLESHOOT



AWS



DOCKER



KUBERNETES



GIT



GITHUB



JENKINS



TERRAFORM



ANSIBLE



PROMETHEUS



GRAFANA



NGINX



REDIS



POSTGRESQL



MONGODB



PYTHON



LINUX



NETWORKING



SECURITY



MONITORING



LOGGING



PRACTICAL
LAB SETUPS



REAL-WORLD
SCENARIOS



TROUBLESHOOT
LIKE A PRO



END-TO-END
WORKFLOWS

LEARN DEVOPS BY **DOING**. BUILD YOUR **SKILLS**. OWN YOUR **SYSTEMS**.

1. HOME LAB ARCHITECTURE

WHAT A SERIOUS DEVOPS HOME LAB SHOULD SIMULATE

- End-to-end DevOps workflows
- Real-world tools and integrations
- CI/CD pipelines, infrastructure, and environments
- Monitoring, logging, alerts, and troubleshooting
- Security, access control, and secrets management
- Failure scenarios and recovery
- Production-like operations and decision making

MANAGEMENT, APPLICATION, AND OBSERVABILITY LAYERS



MANAGEMENT LAYER (CONTROL PLANE)

Tools that manage, automate, and orchestrate the environment.

Examples: Git, Jenkins, Ansible, Terraform



APPLICATION LAYER (DATA PLANE)

Where applications run and deliver value to users.

Examples: Docker, Kubernetes, Databases, Caches, APIs



OBSERVABILITY LAYER (INSIGHT LAYER)

Provides visibility into health, performance, and behavior.

Examples: Prometheus, Grafana, Logs



LOCAL VERSUS CLOUD RESOURCES

LOCAL RESOURCES

VS

CLOUD RESOURCES

- Full control
- No ongoing cost
- Works offline
- Limited by local hardware
- Best for learning, testing, and experimentation

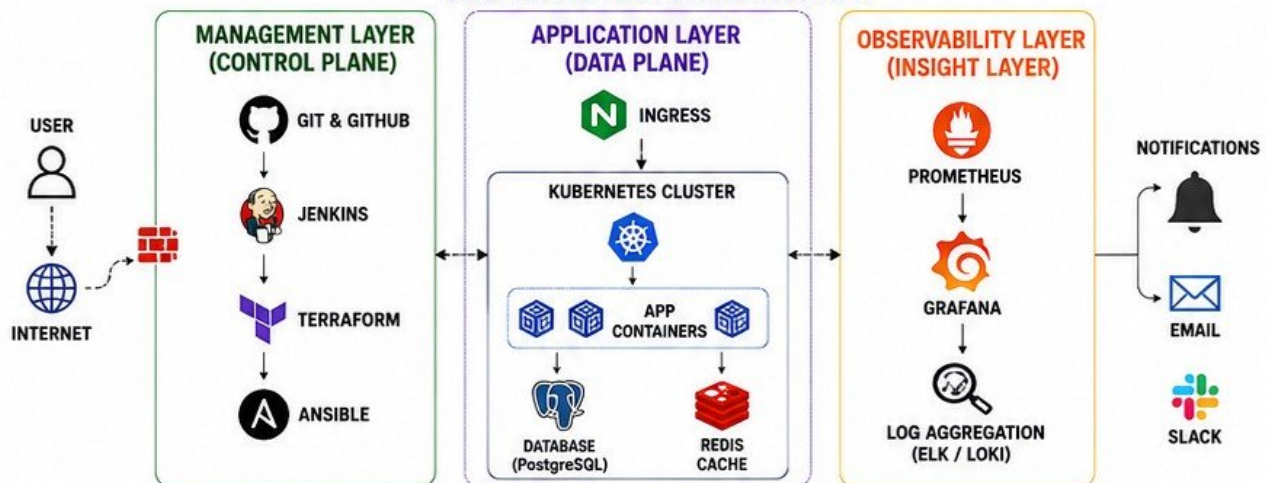
- On-demand scalability
- Pay-as-you-go
- Real cloud experience
- Internet required
- Best for advanced scenarios and hybrid labs



NETWORK BOUNDARIES

- Isolate your lab from your personal network
- Use a private virtual network (e.g., VirtualBox Host-Only, NAT, or custom VPC in cloud)
- Define clear zones: Management, Application, Data, Public
- Control traffic with firewalls and security groups
- Never expose unnecessary services to the internet

LAB ARCHITECTURE DIAGRAM



RESOURCE PLANNING (MINIMUM RECOMMENDED)

CPU	8 vCPU (16+ recommended)
RAM	32 GB (64 GB recommended)
STORAGE	200 GB SSD (500 GB+ recommended)
NETWORK	1 Gbps (Stable connection recommended)
VIRTUALIZATION	16+ GB RAM for VM/Kubernetes workloads



PRODUCTION NOTE

BUILD FOR FAILURE, REBUILDING, AND EXPERIMENTATION.

- Break things on purpose.
- Recover and understand why.
- Document lessons learned.
- Repeat until it becomes natural.



2. WORKSTATION AND LAB ENVIRONMENT



1. LINUX WORKSTATION OR VM

- Use a stable Linux distribution
- Ubuntu LTS recommended
- Works natively or as a VM
- Keep your host OS separate from lab
- This is your control machine



Ubuntu 22.04 LTS
Recommended



2. CPU, RAM, DISK, AND NETWORKING REQUIREMENTS

CPU	8 vCPU (16+ cores recommended)
RAM	32 GB (64 GB recommended)
DISK	200 GB SSD (500 GB+ recommended)
NETWORK	1 Gbps (Stable connection recommended)
IPV4 SPACE	Private IP range for lab networking
IPV6 (OPTIONAL)	Recommended for advanced labs



3. VIRTUALIZATION OPTIONS



VIRTUALBOX

Free, easy to use,
great for most labs



VMWARE WORKSTATION PRO

Powerful features,
snapshots, better
performance



KVM (VIA PROXMOX OR LIBVIRT)

Open source, enterprise
grade, excellent
performance

Enable VT-x/AMD-V in BIOS/UEFI



4. WSL CONSIDERATIONS (WINDOWS USERS)



WSL2

RECOMMENDED

- Install Ubuntu via Microsoft Store
- Enable WSL2 backend
- More performant networking and file system
- Use for lightweight tools and scripting
- Heavy workloads: prefer full VM

ENABLE WSL2 (POWERSHELL AS ADMIN)

```
wsl --install  
wsl --set-default-version 2  
wsl --update
```



5. SSH SETUP

- Generate SSH key pair
- Use key-based authentication
- Disable root login on your servers
- Copy public key to lab servers

SSH KEY GENERATION

```
ssh-keygen -t ed25519  
-C "lab-user"  
ssh-copy-id user@server-ip
```



6. PACKAGE MANAGEMENT

- Keep your system updated
- Use package managers
- Install essential tools
- Automate setup with scripts

UBUNTU (APT)

```
sudo apt update  
sudo apt upgrade -y  
sudo apt install -y \  
git curl vim htop  
net-tools gnupg lsb-release
```



7. DIRECTORY STRUCTURE FOR LAB PROJECTS

```
~/devops-labs  
├── 01-infrastructure  
│   ├── terraform  
│   └── ansible  
├── 02-applications  
│   ├── docker  
│   └── kubernetes  
├── 03-pipelines  
│   ├── jenkins  
│   └── scripts  
├── 04-monitoring  
│   ├── prometheus  
│   └── grafana  
├── 05-logs  
└── elk-stack  
└── docs
```

BEST PRACTICES

- Use clear and consistent naming
- Keep code and configs in version control
- Separate environments (dev, staging, prod)
- Document everything



8. ENVIRONMENT VERIFICATION CHECKLIST

☐	OS VERSION	lsb_release -a
☐	CPU CORES	nproc
☐	MEMORY	free -h
☐	DISK SPACE	df -h
☐	VIRTUALIZATION	systemd-detect-virt
☐	NETWORK CONNECTIVITY	ping 8.8.8.8
☐	SSH ACCESS	ssh user@server-ip
☐	DOCKER (IF INSTALLED)	docker --version
☐	KUBECTL (IF INSTALLED)	kubectl version --client
☐	GIT INSTALLED	git --version



PRODUCTION NOTE

YOUR HOME LAB IS YOUR SANDBOX. BUILD FOR FAILURE, REBUILDING, AND EXPERIMENTATION.



3. LINUX SERVER LAB

BUILD AND MANAGE MULTIPLE LINUX SERVERS



LAB SETUP: CREATE MULTIPLE LINUX SERVERS



WEB-SERVER

OS: Ubuntu 22.04 LTS
IP: 192.168.56.101
ROLE: Web Server



APP-SERVER

OS: Ubuntu 22.04 LTS
IP: 192.168.56.102
ROLE: Application Server



DB-SERVER

OS: Ubuntu 22.04 LTS
IP: 192.168.56.103
ROLE: Database Server



GOAL

- Learn core Linux administration
- Simulate real server operations
- Build strong troubleshooting skills
- Operate like a production engineer

1

CREATE MULTIPLE LINUX SERVERS



- Install Linux (VM or physical)
- Assign static IP addresses
- Set hostname for each server
- Test connectivity between servers

HOSTNAME SETUP

```
sudo hostnamectl set-hostname
web-server
hostnamectl
cat /etc/hostname
```

2

USERS AND GROUPS



- Create users for administration
- Create groups for access control
- Add users to groups
- Set strong passwords

EXAMPLES

```
sudo groupadd devops
sudo useradd -m -G devops
alice
sudo passwd alice
id alice
```

3

SSH KEY AUTHENTICATION



- Generate SSH key pair
- Copy public key to servers
- Disable password auth (optional)
- Test key-based login

KEY SETUP

```
ssh-keygen -t ed25519
ssh-copy-id
user@server-ip
ssh user@server-ip
```

4

SUDO AND PRIVILEGE MANAGEMENT



- Use sudo for admin tasks
- Edit sudoers securely
- Grant least privilege access
- Audit sudo usage

EXAMPLES

```
sudo -l
sudo visudo
alice ALL=(ALL) NOPASSWD:
/usr/bin/systemctl restart
nginx
```

5

FILESYSTEM PERMISSIONS



- Understand user, group, others
- Read (r), Write (w), Execute (x)
- Change ownership with chown
- Modify permissions with chmod

EXAMPLES

```
ls -l /etc/passwd
chmod 750 script.sh
chown alice:devops file.txt
chmod -R 644 /var/www/html
```

6

SYSTEMD SERVICES



- Manage services with systemctl
- Start, stop, restart services
- Enable services at boot
- Check service status

EXAMPLES

```
sudo systemctl start nginx
sudo systemctl status nginx
sudo systemctl enable nginx
sudo systemctl daemon-reload
```

7

PROCESSES AND RESOURCE USAGE



- View running processes
- Check CPU, memory, disk usage
- Monitor system resources
- Identify resource-hungry processes

EXAMPLES

```
top
htop
ps aux --sort=-%cpu
free -h
df -h
```

8

LOGS WITH JOURNALCTL



- View logs for services
- Filter logs by time
- Follow logs in real time
- Investigate issues quickly

EXAMPLES

```
journalctl -u nginx
journalctl -xe
journalctl -u nginx
--since "-1 hour"
journalctl -f
```

9

TROUBLESHOOTING FAILED SERVICES



- Check service status
- Review logs
- Verify configuration
- Check dependencies
- Fix and restart service

TROUBLESHOOTING WORKFLOW

1	Check status	systemctl status <service>
2	Check logs	journalctl -u <service> --no-pager -n 50
3	Check configuration	cat /etc/<service>/<service>.conf
4	Check dependencies	systemctl list-dependencies <service>
5	Fix and restart	systemctl restart <service>

COMMON ISSUES

- Service not running
- Port already in use
- Permission denied
- Configuration error
- Missing dependencies



PRACTICE EVERY DAY. BREAK THINGS. FIX THEM. GET BETTER.



REAL SKILLS COME FROM REAL PRACTICE.

4. DEVOPS NETWORKING LAB

BUILD, CONFIGURE, AND TROUBLESHOOT NETWORKS LIKE A PRO

1 PRIVATE IP ADDRESSING

- Use private IP ranges
- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16
- Assign IPs to servers
- Avoid IP conflicts

EXAMPLE LAB NETWORK

SERVER	IP ADDRESS	ROLE
WEB-SERVER	192.168.10.11	Web
APP-SERVER	192.168.10.12	App
DB-SERVER	192.168.10.13	DB
MONITORING	192.168.10.14	Monitor
JUMP-HOST	192.168.10.10	Bastion

2 DNS (DOMAIN NAME SYSTEM)

- Map names to IP addresses
- Use /etc/hosts for local DNS
- Run a local DNS server (optional)
- Understand forward and reverse DNS

/ETC/HOSTS EXAMPLE

192.168.10.11	web.local
192.168.10.12	app.local
192.168.10.13	db.local
192.168.10.14	monitor.local

3 PORTS AND SOCKETS

- Ports identify services
- Well-known ports:
 - 22 SSH
 - 80 HTTP
 - 443 HTTPS
 - 3306 MySQL
 - 5432 PostgreSQL
- Check open ports
- Understand sockets

COMMON PORTS

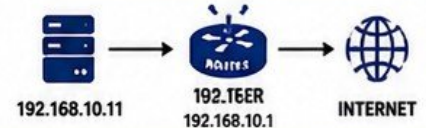
PORT	PROTOCOL	SERVICE
22	TCP	SSH
80	TCP	HTTP
443	TCP	HTTPS
3306	TCP	MySQL
5432	TCP	PostgreSQL
6379	TCP	Redis

4 ROUTING

- Packets need a path
- Use routing tables
- Default gateway sends traffic outside the network
- Understand static and dynamic routing

ROUTE TABLE EXAMPLE

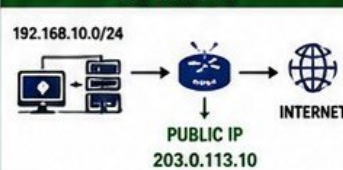
DESTINATION	GATEWAY	INTERFACE
192.168.10.0/24	—	eth0
0.0.0.0/0	192.168.10.1	eth0



5 NAT (NETWORK ADDRESS TRANSLATION)

- Allows private networks to access the internet
- Masquerade (SNAT)
- Port forwarding (DNAT)
- Verify outbound and inbound connectivity

NAT EXAMPLE



6 FIREWALLS

- Control traffic in and out
- Default deny is safest
- Allow only necessary ports
- Use ufw, firewall, or iptables

UFW EXAMPLE

```

sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status
  
```

7 NETWORK DIAGNOSTIC COMMANDS

PING
Test reachability to a host

```
ping 192.168.10.11
```

DIG
Query DNS records

```
dig web.local
```

SS
Show sockets and connections

```
ss -tulnp
```

CURL
Test HTTP/HTTPS connectivity

```
curl -I http://web.local
```

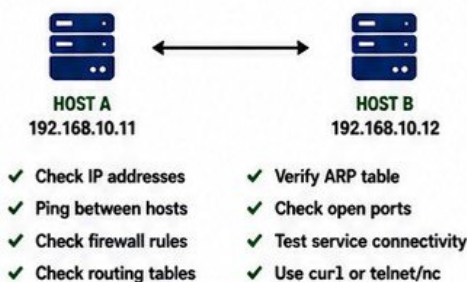
TRACEROUTE
Trace the path to a destination

```
traceroute 8.8.8.8
```

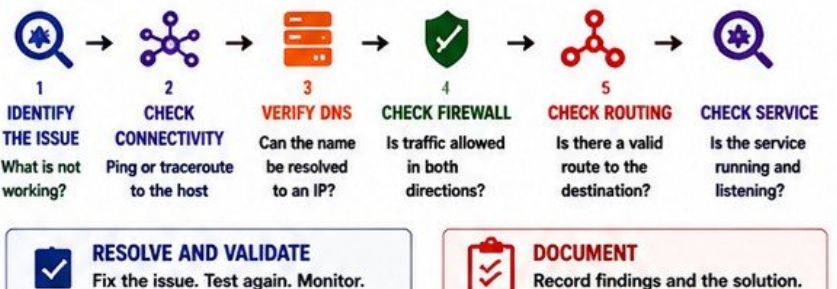
NMAP (OPTIONAL)
Advanced network scanning

```
nmap -sS 192.168.10.0/24
```

8 DIAGNOSE HOST-TO-HOST CONNECTIVITY



9 NETWORK TROUBLESHOOTING FLOW



PRODUCTION NOTE

NETWORKS FAIL. GOOD ENGINEERS DIAGNOSE, UNDERSTAND, AND BUILD RESILIENT SYSTEMS.



5. GIT AND GITHUB WORKFLOW LAB

VERSION CONTROL IS THE FOUNDATION OF DEVOPS

1 REPOSITORY STRUCTURE

```

my-project/
├── README.md
├── .gitignore
├── LICENSE
├── src/
├── tests/
├── docs/
└── scripts/
  
```

BEST PRACTICES

- Keep it simple and organized
- Separate source, tests, docs, configs
- Use meaningful file and folder names
- Document everything

2 BRANCHING



- main: Production-ready code
- feature/*: New features
- bugfix/*: Bug fixes
- Never commit directly to main

3 PULL REQUESTS



- Propose changes
- Discuss and review
- Automated checks run
- Approve and merge
- Keeps code quality high

PR CHECKLIST

- ✓ Clear title and description
- ✓ Linked issue
- ✓ Tests added/updated
- ✓ Code reviewed
- ✓ No merge conflicts
- ✓ CI checks passing

4 MERGE STRATEGIES

STRATEGY	WHEN TO USE
Merge Commit	Preserve full history
Squash and Merge	Keep history clean
Rebase and Merge	Linear history
Fast Forward	No extra merge commit

RECOMMENDATION

Use Squash Merge for feature branches to keep history simple.



5 TAGS AND RELEASES



TAGS

- Mark important points
- Example: v1.0.0
- Immutable reference



RELEASES

- Bundle changes
- Add release notes
- Attach binaries (optional)
- Announce to team/users

6 .GITIGNORE



COMMON ENTRIES

```

node_modules/
.env
*.log
*.tmp
build/
dist/
.DS_Store
coverage/
  
```

TIP

Keep sensitive files and build artifacts out of Git.

7 PROTECTED BRANCHES



- Protect main and release branches
- Require pull requests
- Require review before merge
- Require status checks to pass
- Restrict who can push

EXAMPLE RULES FOR MAIN

- ✓ Require pull request reviews
- ✓ Require status checks
- ✓ Prevent force pushes
- ✓ Dismiss stale reviews
- ✓ Include administrators

8 GITHUB AUTHENTICATION



HTTPS (RECOMMENDED)

Use Personal Access Token (PAT) instead of password

`https://token@github.com/user/repo.git`



SSH (SECURE)

- Generate SSH key
- Add public key to GitHub
- Clone using SSH URL

`git@github.com:user/repo.git`

9 TEAM DEVELOPMENT WORKFLOW SIMULATION



- Developer creates a feature branch
- Developer writes code and commits
- Push branch to GitHub
- Open Pull Request
- Reviewer reviews and approves
- Merge to main branch
- CI/CD pipeline runs and deploys

SIMULATION IDEAS

- Create multiple users
- Work on different features
- Open PRs and review each other's code
- Introduce conflicts and resolve
- Merge and deploy changes
- Tag a release

COMMON GIT COMMANDS

<code>> git clone <repo-url></code>	Clone repository	<code>> git pull origin <branch></code>	Pull latest changes
<code>> git status</code>	Check status	<code>> git branch</code>	List branches
<code>> git add .</code>	Stage changes	<code>> git checkout <branch></code>	Switch branches
<code>> git commit -m "message"</code>	Commit changes	<code>> git log --oneline --graph</code>	View history
<code>> git push origin <branch></code>	Push changes	<code>> git tag -a v1.0.0 -m "Release"</code>	Create annotated tag

GIT BEST PRACTICES

- ✓ Commit small and often
- ✓ Write meaningful commit messages
- ✓ Pull before you push
- ✓ Review before you merge
- ✓ Keep main branch deployable
- ✓ Document your workflow



PRODUCTION NOTE

A STRONG GIT WORKFLOW = CLEAN CODE, HAPPY TEAM, AND RELIABLE DEPLOYMENTS.



6. DOCKER HOME LAB

BUILD, RUN, AND MANAGE CONTAINERS LIKE A PRO



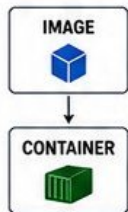
1 DOCKER ENGINE



- Install Docker Engine on Linux
- Start and enable service
- Verify installation
- Docker daemon configuration
- docker info, docker version

```
sudo systemctl enable --now docker
docker run hello-world
```

2 IMAGES AND CONTAINERS



- Images are read-only templates
- Containers are running instances
- Create, start, stop, restart containers
- List and inspect containers

```
docker run -d --name web nginx
docker ps -a
```

3 DOCKERFILES



- Define instructions to build images
- FROM, RUN, COPY, CMD, EXPOSE, ENV
- Keep images small and secure
- Use .dockerignore

```
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y \
  curl
CMD ["/bin/bash"]
```

4 MULTI-STAGE BUILDS



- Build with multiple stages
- Keep final image small
- Copy only what is needed
- Reduce attack surface

```
FROM node:20 AS build
WORKDIR /app
COPY . .
RUN npm install && npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

5 VOLUMES



- Persist data outside the container
- Named volumes recommended
- Bind mounts for development
- Backup and reuse data

```
docker volume create app_data
docker run -d -v app_data:/data mysql:8
docker run -v $(pwd):/app -it node:20 bash
```

6 NETWORKS



- Containers communicate over networks
- Default bridge network
- Create custom networks
- Isolate environments

```
docker network create app_net
docker run -d --name backend --network app_net api
docker run -d --name frontend --network app_net web
```

7 ENVIRONMENT VARIABLES



- Pass configuration to containers
- Keep secrets out of images
- Use -e or --env-file
- Use .env files for convenience

```
docker run -d --name app \
  -e DB_HOST=db \
  --env-file .env \
  myapp:latest
```

8 HEALTH CHECKS



- Monitor container health
- Docker restarts unhealthy containers
- Use in production always

```
HEALTHCHECK --interval=30s --timeout=5s \
  --start-period=10s --retries=3 \
  CMD curl -f http://localhost/health || exit 1
```

9 CONTAINER LOGS



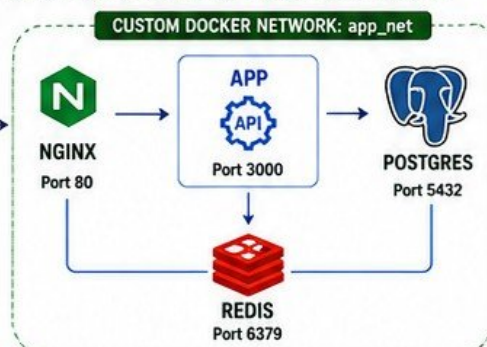
- View container logs
- Follow logs in real time
- Use timestamps
- Redirect logs if needed

```
docker logs web
docker logs -f web
docker logs --since 10m web
```

10 BUILD AND RUN A PRODUCTION-STYLE APPLICATION STACK

STACK OVERVIEW

- NGINX (Web)**
Serves frontend
- APP (Backend API)**
Business logic
- POSTGRES (Database)**
Stores data
- REDIS (Cache)**
Improves performance



RUN THE STACK

```
# Create network
docker network create app_net

# Database
docker run -d --name db --network app_net \
  -e POSTGRES_PASSWORD=pass postgres:15

# Redis
docker run -d --name redis --network app_net redis:7

# Backend API
docker run -d --name api --network app_net \
  -e DB_HOST=db -e REDIS_HOST=redis myapi:latest

# Nginx (Frontend)
docker run -d --name web -p 80:80 --network app_net \
  -v $(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro nginx:alpine
```

BEST PRACTICES

- ☒ Keep images small
- ☒ Use multi-stage builds
- ☒ Scan images for vulnerabilities
- ☐ Don't run as root
- ☐ Use specific tags
- ☐ Clean up unused images and volumes

USEFUL COMMANDS CHEAT SHEET

- | | | | |
|--|---------------------|--|----------------------|
| <code>docker ps -a</code> | List all containers | <code>docker rmi <id></code> | Remove image |
| <code>docker images</code> | List images | <code>docker system prune -a</code> | Clean up |
| <code>docker inspect <id></code> | Inspect details | <code>docker exec -it <id> bash</code> | Connect to container |
| <code>docker rm -f <id></code> | Remove container | | |



PRODUCTION NOTE

LEARN BY DOING. BREAK CONTAINERS. CHECK LOGS.
FIX ISSUES. REPEAT. THAT'S HOW YOU GET BETTER.



7. DOCKER COMPOSE APPLICATION STACK



1 FRONTEND



- Nginx serving static files
- Port: 80
- Connects to Backend API

2 BACKEND API



- REST API (Node.js / Python / Java / Go)
- Port: 5000
- Connects to Database and Redis

3 DATABASE



- PostgreSQL
- Port: 5432
- Persistent data with volume

4 REDIS



- Caching and session store
- Port: 6379
- Improves API performance

5 PERSISTENT STORAGE



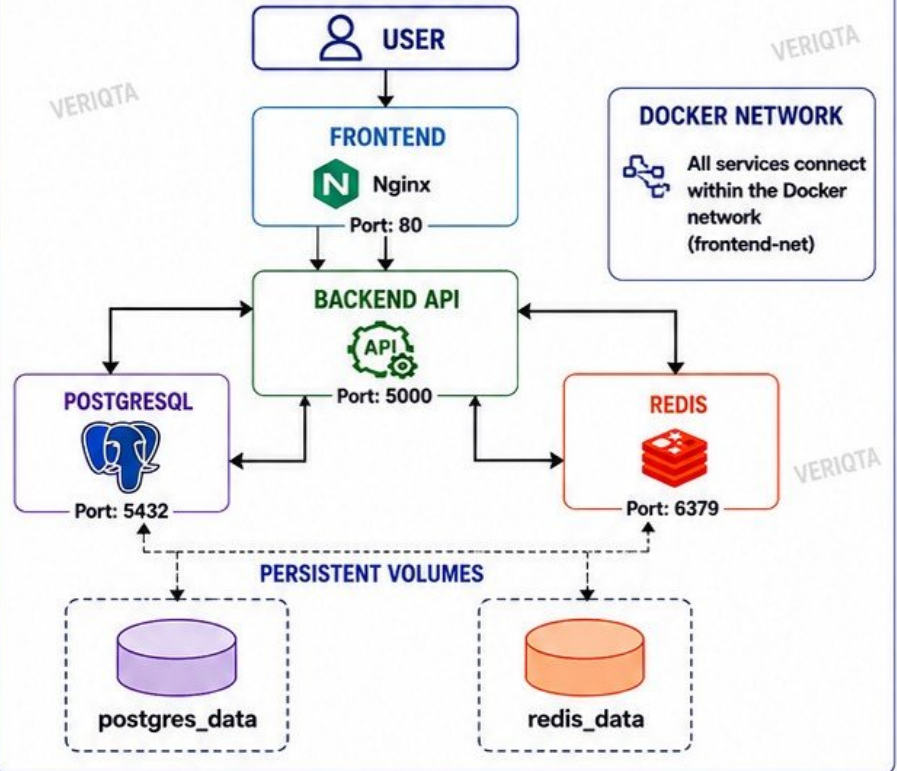
- All services run on the same Docker network
- Use service names as hostnames
- Example: backend connects to postgresql:5432
- No external exposure between containers

8 DEPENDENCIES



- Define startup order using depends_on
- Wait for health checks to pass
- Example: Backend waits for Database and Redis
- Avoid race conditions at startup

APPLICATION STACK DIAGRAM



7 HEALTH CHECKS



- Ensure containers are ready before use
- Example health checks:
 - ✓ Frontend: `curl -f http://localhost`
 - ✓ Backend: `curl -f http://localhost:5000/health`
 - ✓ Database: `pg_isready`
 - ✓ Redis: `redis-cli ping`
- Prevents traffic to unhealthy containers

9 RESTART POLICIES



- Always restart containers unless stopped manually
- Recommended: `unless-stopped`
- Ensures high availability in your lab
- Example for API, DB, Redis, Frontend

10 TROUBLESHOOT CONTAINER-TO-CONTAINER COMMUNICATION



COMMON ISSUES

- Wrong service name or network
- Port not exposed (internal or external)
- Service not ready (startup delay)
- Firewall or security group (if using cloud)
- Application misconfiguration

TROUBLESHOOTING STEPS

- 1 Check if containers are running: `docker ps`
- 2 Inspect network: `docker network inspect frontend-net`
- 3 Exec into a container: `docker exec -it <container> sh`
- 4 Test connectivity: `ping <service>` or `curl http://<service>:<port>`
- 5 Check logs: `docker logs <container>`
- 6 Verify service health endpoints



USEFUL COMMANDS

- `docker ps`
- `docker compose ps`
- `docker logs <service>`
- `docker exec -it <service> sh`
- `docker inspect <service>`
- `docker compose config`

8. JENKINS CI LAB



Jenkins

1 JENKINS SERVER INSTALLATION



- Run Jenkins in Docker
- Port: 8080
- Persist data with a volume
- Initial admin setup and plugins

2 AGENTS



- Add any machine as an agent
- Run builds in parallel
- Label agents for workload
- Use JNLP or SSH agents

3 CREDENTIALS



- Store secrets securely
- Types: Username/Password, SSH, Secret Text, Tokens
- Use credentials in pipelines

4 PIPELINE AS CODE



- Define pipelines in code (Jenkinsfile)
- Version controlled with your app
- Reproducible and auditable
- Supports Declarative & Scripted

5 JENKINSFILE



- Stored in your Git repository
- Defines pipeline structure
- Easy to review and maintain
- Example on the right

6 BUILD STAGES



- Define stages: Checkout, Build, Test, Package, Deploy
- Execute steps in order
- Fail fast on errors

7 TESTING



- Run unit and integration tests
- Fail the build on test failure
- Publish test reports (JUnit, etc.)
- Quality gates and thresholds

8 ARTIFACTS



- Archive build outputs
- Store JARs, WARs, images, etc.
- Use for deployments
- Keep artifacts for traceability

9 PIPELINE LOGS



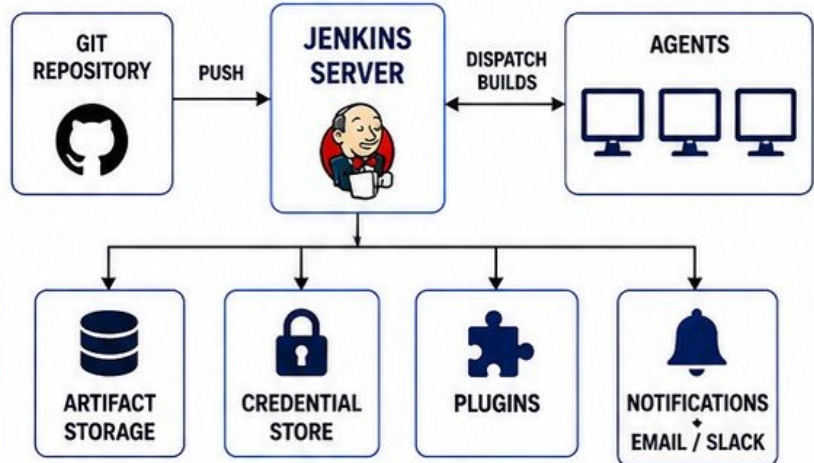
- Console output for each step
- Color-coded logs
- Easy to navigate and search
- Download or share logs

10 TRIGGER BUILDS FROM GIT



- Webhook triggers on push
- Build on branch, tag, or PR
- Poll SCM (backup option)
- Automatic and fast feedback

JENKINS CI ARCHITECTURE



EXAMPLE JENKINSFILE (DECLARATIVE PIPELINE)

```

pipeline {
  agent any
  environment {
    NODE_ENV = 'test'
  }
  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
    stage('Build') {
      steps {
        sh 'mvn clean package -DskipTests'
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
    stage('Package') {
      steps {
        archiveArtifacts artifacts: 'target/*.jar'
      }
    }
  }
  post {
    always {
      junit 'target/surefire-reports/*.xml'
    }
  }
}
  
```

BENEFITS OF JENKINS CI

- ✓ Automate repetitive tasks
- ✓ Early feedback on changes
- ✓ Integrates with DevOps tools
- ✓ Consistent and reliable builds
- ✓ Scalable with agents
- ✓ Full visibility and traceability

BEST PRACTICES

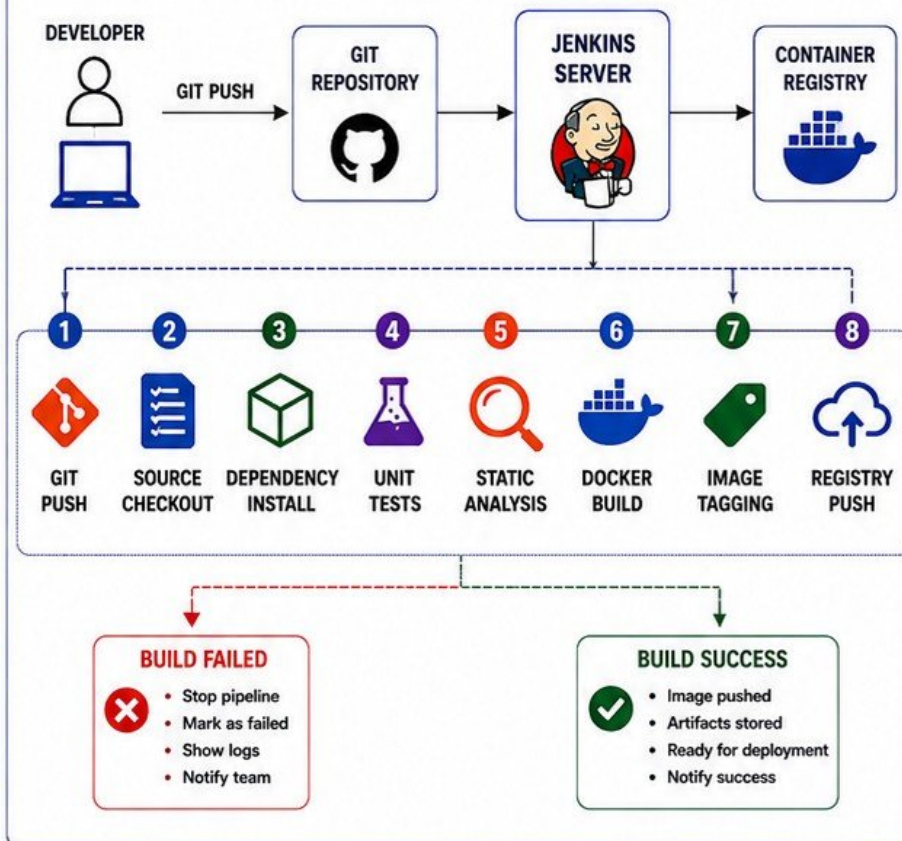
- ★ Keep Jenkinsfile in Git
- ★ Use agents for scale
- ★ Secure your credentials
- ★ Fail fast and fix early
- ★ Keep pipelines simple and modular

9. BUILD A COMPLETE CI PIPELINE

FROM CODE COMMIT TO CONTAINER REGISTRY

- 1 GIT PUSH**
 - Developer commits code
 - Pushes to remote repository
 - Triggers the CI pipeline
- 2 SOURCE CHECKOUT**
 - Pipeline checks out the source code
 - Uses the latest commit
 - Clean workspace prepared
- 3 DEPENDENCY INSTALLATION**
 - Install required dependencies
 - Use lock files for consistency
 - Cache dependencies for speed
- 4 UNIT TESTS**
 - Run unit tests
 - Generate test reports
 - Fail the build if tests fail
- 5 STATIC ANALYSIS**
 - Run code quality checks
 - Detect bugs, vulnerabilities, and smells
 - Enforce quality gates
- 6 DOCKER IMAGE BUILD**
 - Build Docker image from Dockerfile
 - Use multi-stage builds if applicable
 - Fail build if image build fails
- 7 IMAGE TAGGING**
 - Tag image with version
 - Use commit SHA and 'latest'
 - Example: myapp:1.0.0, myapp:abc123
- 8 REGISTRY PUSH**
 - Authenticate to container registry
 - Push tagged image
 - Verify image is available
- 9 FAILED-BUILD HANDLING**
 - Stop pipeline immediately
 - Mark build as failed
 - Send notifications
 - Provide actionable logs
 - No artifact or image is published
- 10 SUCCESS**
 - Pipeline completes successfully
 - Image is available in registry
 - Ready for deployment stage

CI PIPELINE ARCHITECTURE DIAGRAM



PIPELINE LOGS AND VISIBILITY

- View logs for each stage
- Color-coded output (Success / Failure / Running)
- Search and filter logs
- Download logs and artifacts
- Replay or rebuild pipeline

```

Pipeline
Started by user dev
Running on Jenkins in /var/jenkins/workspace/myapp

[1] Git Push          SUCCESS    2s
[2] Source Checkout   SUCCESS    3s
[3] Dependency Install SUCCESS   15s
[4] Unit Tests        SUCCESS    8s
[5] Static Analysis   SUCCESS    6s
[6] Docker Build      SUCCESS   30s
[7] Image Tagging     SUCCESS    2s
[8] Registry Push     SUCCESS   12s

Finished: SUCCESS    Total time: 78s
  
```

BEST PRACTICES



Keep Jenkinsfile
in Git



Make builds
reproducible



Secure credentials
and secrets



Fail fast and
fix early



Use caching to
speed up builds



Always review
pipeline logs

10. TERRAFORM INFRASTRUCTURE LAB



Terraform

1 TERRAFORM PROJECT STRUCTURE



- Keep your code organized and modular
- Standard structure for all projects
- Easy to maintain and scale

```
my-infrastructure/
├── main.tf
├── variables.tf
├── outputs.tf
├── providers.tf
├── terraform.tfvars
├── modules/
│   └── <module-name>/
│       ├── main.tf
│       └── README.md
```

2 PROVIDERS



- Define the cloud or platform
- Configure provider settings
- Example: AWS Provider

```
provider "aws" {
  region = "us-east-1"
  profile = "default"
}
```

3 VARIABLES



- Make your code dynamic
- Input values at runtime
- Store in terraform.tfvars

```
variable "instance_type" {
  description = "EC2 type"
  type        = string
  default     = "t3.micro"
}
```

4 OUTPUTS



- Expose important values
- Use outputs for information sharing
- Consume in other modules

```
output "instance_ip" {
  description = "Public IP"
  value       = aws_instance
    .web.public_ip
}
```

5 RESOURCES



- Define the infrastructure
- What Terraform will create
- Example: EC2 Instance

```
resource "aws_instance" "web" {
  ami      = "ami-0abcdef1234567890"
  instance_type = var.instance_type
  tags = {
    Name = "web-server"
  }
}
```

6 DATA SOURCES



- Fetch information from existing resources
- Use to reference values
- Example: VPC, AMI

```
data "aws_vpc" "default" {
  default = true
}
```

7 STATE



- Terraform tracks real infrastructure in the state
- Stored locally or remotely
- Never edit the state file manually

```
terraform.tfstate
(Tracks real world infrastructure)
```

8 PLAN AND APPLY WORKFLOW



WRITE CODE



terraform plan



REVIEW PLAN



terraform apply

WORKFLOW COMMANDS



- terraform init # Initialize the working directory
- terraform plan # Preview the changes
- terraform apply # Apply the changes
- terraform show # Show current state
- terraform validate # Validate configuration

9 DESTROY AND REBUILD INFRASTRUCTURE



DESTROY INFRASTRUCTURE

terraform destroy

- Removes all resources
- Confirm before destroy
- Clean state after destroy



REBUILD INFRASTRUCTURE

terraform apply

- Recreates all resources
- Uses the same code
- Infrastructure as Code power

10 INTRODUCE CONFIGURATION DRIFT DELIBERATELY

APPLY INFRASTRUCTURE



MAKE MANUAL CHANGE



(Outside Terraform)
Example: Change Security Group Rule in AWS Console

DETECT DRIFT



terraform plan shows changes that are not in your code

WHY THIS MATTERS

- ✓ Understand how drift happens in real environments
- ✓ Learn to detect and correct drift
- ✓ Keep your infrastructure consistent and reliable

TERRAFORM INFRASTRUCTURE EXAMPLE (AWS)

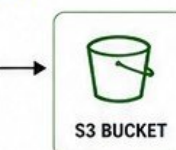


VPC



PUBLIC SUBNET

EC2 INSTANCE

PRIVATE SUBNET
RDS DATABASE
(PostgreSQL)

S3 BUCKET



ROUTE 53



KEY TAKEAWAY

Terraform allows you to define, provision, manage, and destroy infrastructure safely and repeatably. Practice, break things, and rebuild until you understand every step.

11. ANSIBLE CONFIGURATION MANAGEMENT LAB



1 CONTROL NODE



- The machine where Ansible is installed and run
- Initiates connections to managed nodes via SSH

2 MANAGED NODES



- Target servers that Ansible manages
- No agent installation required

3 INVENTORY



- Defines groups and hosts
- Static (INI) or Dynamic (Plugin)
- Stored in inventory.ini (or inventory directory)

4 SSH CONNECTIVITY



- Passwordless SSH access
- Use SSH keys for security
- Verify with: `ansible <host> -m ping`

5 PLAYBOOKS



- YAML files that define tasks
- Executed in order on target hosts
- Idempotent by design

6 ROLES



- Reusable, shareable task bundles
- Standard directory structure
- Promote DRY and consistency

7 VARIABLES



- Store values for reuse
- Defined in vars files, group_vars, host_vars, or inline
- Support precedence

8 TEMPLATES



- Jinja2 templates for dynamic files
- Render files with variables
- Example uses: configs, scripts, HTML, system files

9 HANDLERS



- Run tasks only when notified
- Common use: restart services
- Prevents unnecessary restarts

10 IDEMPOTENCY



- Ansible ensures desired state
- Re-running playbooks makes no changes if already compliant
- Safe to run repeatedly

ANSIBLE ARCHITECTURE

CONTROL NODE (ANSIBLE)



- PLAYBOOKS
- INVENTORY
- MODULES
- PLUGINS
- ROLES
- VARIABLES
- TEMPLATES
- HANDLERS

SSH

MANAGED NODES



WEB SERVER 1 (UBUNTU)



WEB SERVER 2 (CENTOS)



DATABASE SERVER (ROCKY LINUX)

EXAMPLE INVENTORY (INI FORMAT)

```
[webservers]
web1 ansible_host=192.168.56.11
web2 ansible_host=192.168.56.12

[dbservers]
db1 ansible_host=192.168.56.20

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

EXAMPLE PLAYBOOK (SITE.YML)

```
---
- name: Configure Web Servers
  hosts: webservers
  become: yes
  vars:
    package_name: nginx
  tasks:
    - name: Install Nginx
      apt:
        name: "{{ package_name }}"
        state: present
    - name: Copy Nginx Config
      template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        notify: Restart Nginx
  handlers:
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
```

KEY FEATURES

- ✓ Agentless architecture (uses SSH)
- ✓ Simple YAML syntax
- ✓ Powerful modules
- ✓ Highly extensible
- ✓ Repeatable and reliable automation

HOW IT WORKS



Run playbook from control node



Connect to managed nodes via SSH



Execute tasks in order



Ensure desired state (idempotent)



Report status and changes

CONFIGURE MULTIPLE LINUX SERVERS AUTOMATICALLY



WRITE PLAYBOOK



DEFINE INVENTORY



RUN PLAYBOOK



APPLY CONFIGURATIONS TO ALL NODES



ALL SERVERS IN DESIRED STATE



PRO TIP

Start small, test often, and build reusable roles.

Automation is not just about saving time, it's about consistency and reliability.

12. KUBERNETES HOME LAB



kubernetes

1 BUILD A LOCAL KUBERNETES CLUSTER



- Use Kind, K3s, or Minikube
- Single node or multi-node setup
- Simulates real production

2 CONTROL PLANE AND WORKER NODES



- Control plane runs core Kubernetes services
- Worker nodes run your applications
- Communicate securely inside the cluster

3 KUBECTL



- Command line tool for Kubernetes
- Interact with the cluster API
- Deploy, inspect and manage resources

4 NAMESPACES



- Virtual clusters inside the same cluster
- Isolate environments
- Example: dev, test, staging, prod

5 PODS



- Smallest deployable unit
- One or more containers
- Share network and storage
- Ephemeral by design

6 DEPLOYMENTS



- Manage replica sets of Pods
- Declarative updates
- Rolling updates and rollbacks
- Self-healing

7 SERVICES



- Stable network endpoint for Pods
- Types: ClusterIP, NodePort, LoadBalancer, ExternalName
- Service discovery and load balancing

CONFIGMAP EXAMPLE

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: dev
data:
  APP_MODE: "production"
  LOG_LEVEL: "info"
```

SECRET EXAMPLE

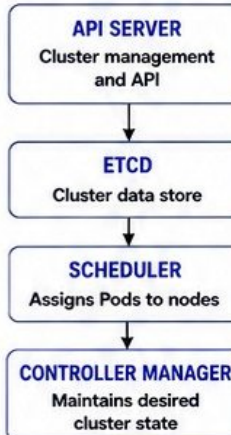
```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
  namespace: dev
type: Opaque
data:
  username: YWRtaW4= # base64
  password: cGFzc3dvcmQ= # base64
```

GET STARTED (SUGGESTED STEPS)

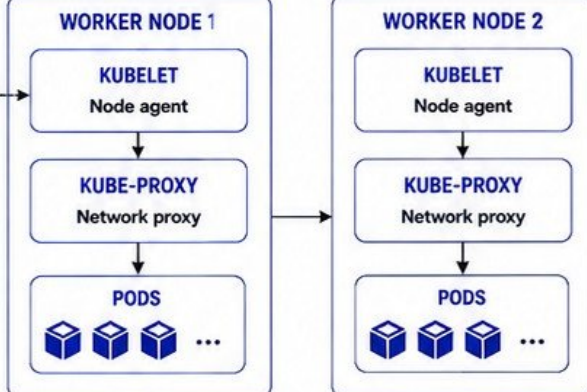
- 1 Install Kind, K3s, or Minikube
- 2 Create a cluster
- 3 Verify nodes: `kubectl get nodes`
- 4 Create a namespace
- 5 Deploy a sample application
- 6 Expose it with a Service
- 7 Explore logs and scaling
- 8 Break things and fix them

KUBERNETES CLUSTER ARCHITECTURE

CONTROL PLANE (MASTER NODE)



WORKER NODES 2



CONTAINER NETWORK INTERFACE (CNI) Provides networking between Pods across nodes



DATA FLOW



COMMON KUBECTL COMMANDS

```
> kubectl get nodes # List all nodes
> kubectl get pods -A # List all Pods in all namespaces
> kubectl get namespaces # List all namespaces
> kubectl get deployments -A # List all Deployments
> kubectl get svc -A # List all Services
> kubectl describe pod <pod-name> # Describe a Pod
> kubectl logs <pod-name> # View Pod logs
> kubectl exec -it <pod-name> -- /bin/bash # Shell into a Pod
> kubectl apply -f <file.yaml> # Create/Update resources
> kubectl delete -f <file.yaml> # Delete resources
```

KEY CONCEPTS

- ✓ Declarative: Define desired state, Kubernetes makes it real
- ✓ Self-Healing: Automatically restarts failed Pods
- ✓ Scalable: Scale up or down with ease
- ✓ Resilient: Designed for high availability
- ✓ Portable: Run anywhere



PRODUCTION NOTE:

Kubernetes is powerful but complex. Learn by building, breaking, troubleshooting, and understanding how each component works together.

13. DEPLOY AN APPLICATION TO KUBERNETES

FROM IMAGE TO A RUNNING, RELIABLE, AND SCALABLE APP

OBJECTIVE

- Deploy a containerized application
- Expose it inside the cluster
- Configure application settings
- Protect sensitive data
- Ensure reliability and availability
- Enable safe updates and rollbacks



1

CONTAINER IMAGE



- Build and push your image to a registry.
- Use a version tag for reproducibility.

```
docker build -t myapp:1.0.0 .
docker push myregistry/myapp:1.0.0
```

2

DEPLOYMENT MANIFEST



- Defines desired state for your application.
- Manages Pods and replica count.

```
deployment.yaml
```

3

SERVICE



- Exposes your Pods to other services or users.
- Use ClusterIP for internal access or NodePort/LoadBalancer for external.

```
service.yaml
```

4

CONFIGMAP



- Store non-sensitive configuration as key-value pairs.
- Keep configs outside the container image.

```
configmap.yaml
```

5

SECRET



- Store sensitive data like passwords, tokens, and keys.
- Mounted as environment variables or files.

```
secret.yaml
```



ROLLING UPDATES

- Update your application without downtime.
- Kubernetes gradually replaces old Pods with new ones.
- Configurable strategy and max unavailable.

```
kubectl set image deployment/myapp myapp=myregistry/myapp:1.1.0
kubectl rollout status deployment/myapp
```

6

RESOURCE REQUESTS AND LIMITS



- Requests = minimum guaranteed resources.
- Limits = maximum allowed resources.
- Prevents noisy neighbor and OOM kills.

```
resources:
  requests:
    cpu: "200m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

7

LIVENESS PROBE



- Checks if the application is running.
- Kubernetes restarts the container if it fails.

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10
```

8

READINESS PROBE



- Checks if the application is ready to serve traffic.
- Pods failing readiness are removed from Service.

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
```

10

ROLLBACKS



- Revert to a previous working version quickly.
- Kubernetes rolls back to the last known good ReplicaSet.

```
kubectl rollout undo deployment/myapp
kubectl rollout history deployment/myapp
```

DEPLOYMENT FLOW SUMMARY



USEFUL COMMANDS

```
kubectl apply -f <file.yaml> | Apply Kubernetes manifest
kubectl get all | View all resources
kubectl describe <resource> <name> | Describe a resource
```

```
kubectl logs <pod-name> | View Pod logs
kubectl exec -it <pod-name> -- sh | Access Pod shell
kubectl delete -f <file.yaml> | Delete resources
```


14. KUBERNETES TROUBLESHOOTING LAB

BREAK, OBSERVE, DIAGNOSE, AND FIX



LAB GOAL

BREAK THE CLUSTER INTENTIONALLY AND DIAGNOSE LIKE A PRODUCTION ENGINEER.

BREAK THE CLUSTER INTENTIONALLY AND DIAGNOSE

1 CRASHLOOPBACKOFF



- Container starts and crashes repeatedly.
- Check logs for errors in the application.

COMMON CAUSES

App misconfiguration, missing files, bad command.

CHECK

`kubectl logs <pod>`

2 IMAGEPULLBACKOFF



- Kubernetes cannot pull the container image.
- Check image name, tag, and registry access.

COMMON CAUSES

Wrong image name, private registry, missing credentials.

CHECK

`kubectl describe pod <pod>`

3 PENDING PODS



- Pod is not scheduled to any node.
- Check resources, taints, node selector, and PVC issues.

COMMON CAUSES

Insufficient resources, storage, taints, affinity.

CHECK

`kubectl describe pod <pod>`

4 FAILED READINESS PROBES



- Pod is running but not ready to serve traffic.
- Check probe path, port, and timing.

COMMON CAUSES

Wrong path, slow app, port mismatch.

CHECK

`kubectl describe pod <pod>`

5 INCORRECT SERVICES



- Service cannot route traffic to Pods.
- Check selector, ports, and endpoints.

COMMON CAUSES

Wrong selector, port mismatch, no endpoints.

CHECK

`kubectl get svc,ep -o wide`

6 DNS FAILURES



- Pods cannot resolve service names.
- Check CoreDNS and network policies.

COMMON CAUSES

CoreDNS down, network policy blocking DNS.

CHECK

`kubectl run dns-test --rm -it --image=busybox -- nslookup kubernetes.default.svc`

7 CONFIGURATION ERRORS



- Invalid or missing configuration.
- Check ConfigMap, Secret, and values.

COMMON CAUSES

Syntax errors, wrong keys, missing mounts.

CHECK

`kubectl describe pod <pod>`

8 RESOURCE EXHAUSTION



- CPU or memory limits are exceeded.
- Check usage and set proper limits.

COMMON CAUSES

Low limits, memory leaks, high traffic.

CHECK

`kubectl top pods,nodes`

9 NODE OR STORAGE ISSUES



- Node not Ready or storage unavailable.
- Check node status and PVC events.

COMMON CAUSES

Node down, disk full, storage class issues.

CHECK

`kubectl get nodes,pvc,sc`

10 NETWORK OR POLICY BLOCKS



- NetworkPolicy or firewall blocking traffic.
- Check policies and rules.

COMMON CAUSES

Deny rules, missing allow rules, wrong CIDR.

CHECK

`kubectl get netpol -A`

ESSENTIAL DEBUGGING COMMANDS

PURPOSE	COMMAND
List all Pods	<code>kubectl get pods -A</code>
Describe a Pod	<code>kubectl describe pod <pod> -n <ns></code>
View Logs	<code>kubectl logs <pod> -n <ns></code>
View Previous Logs	<code>kubectl logs <pod> -n <ns> -p</code>
Exec into Pod	<code>kubectl exec -it <pod> -n <ns> -- sh</code>
View Events	<code>kubectl get events -A --sort-by=.lastTimestamp</code>
View Nodes	<code>kubectl get nodes -o wide</code>
Check Resources	<code>kubectl top pods,nodes -A</code>
View Services	<code>kubectl get svc,ep -A -o wide</code>
Describe Deployment	<code>kubectl describe deploy <name> -n <ns></code>

EVENTS AND ROOT-CAUSE ANALYSIS

- OBSERVE THE SYMPTOM**
Check Pod status, logs, and events.
- COLLECT EVIDENCE**
Use logs, describe, top, and exec.
- IDENTIFY THE FAILURE DOMAIN**
Pod, Node, Network, Storage, or App.
- FIND THE ROOT CAUSE**
Look for the first failing point.
- APPLY THE FIX**
Correct configuration or resources.
- VERIFY AND MONITOR**
Ensure the issue is fully resolved.

EVENTS TELL THE STORY

TYPE	WHAT IT MEANS
Scheduled	Pod assigned to a node.
Pulled	Image pulled successfully.
Created	Container created.
Started	Container started.
BackOff	Container failed and Kubernetes is backing off.
Unhealthy	Probe failed.
FailedMount	Volume mount failed.
FailedScheduling	Pod cannot be scheduled.
Killing	Container is being terminated.
Failed	Container failed to start.

PRODUCTION PRINCIPLE



IN PRODUCTION, YOU WILL NOT HAVE TIME. YOU MUST KNOW WHAT TO CHECK AND IN WHAT ORDER.

- Start with symptoms.
- Collect facts, not guesses.
- Use events, logs, and metrics.
- Change one thing at a time.
- Confirm the fix and prevent recurrence.

TROUBLESHOOTING FLOW



OBSERVE THE ISSUE



COLLECT EVIDENCE



IDENTIFY ROOT CAUSE



APPLY FIX



VERIFY & MONITOR



PREVENT RECURRENCE



LAB EXERCISE IDEAS

- Break a pod by changing the command.
- Use wrong image name.
- Remove ConfigMap key.
- Set CPU limit too low.
- Block DNS with NetworkPolicy.
- Remove Service selector.
- Fill disk on a node.
- Kill kubelet service.
- Watch and fix each issue.

15. KUBERNETES INGRESS AND APPLICATION ROUTING

ROUTE TRAFFIC TO YOUR SERVICES THE RIGHT WAY

OBJECTIVE

- Expose applications safely
- Route traffic intelligently
- Secure with TLS
- Understand request flow
- Build production-like routing

1 INGRESS CONTROLLER



- Monitors Ingress resources.
- Implements routing rules.
- Handles SSL/TLS termination.
- Popular: NGINX Ingress, Traefik, HAProxy.

```
kubectl get ingressclass
```

2 INGRESS RESOURCES



- Defines external access rules.
- Maps URLs to Services.
- Uses Ingress Controller to apply rules.

```
kubectl get ingress -A
```

3 HOST-BASED ROUTING



- Route based on domain names.
- Multiple apps on one IP.
- Each host maps to a Service.

```
host: app.example.com
```

4 PATH-BASED ROUTING



- Route based on URL paths.
- Multiple paths for the same host.
- Each path maps to a Service.

```
path: /api or /web
```

5 INTERNAL SERVICES



- ClusterIP Service exposes Pods inside the cluster.
- Ingress routes traffic to these Services.
- Services handle load balancing to Pods.

```
kubectl get svc
```

6 DNS MAPPING



- Point your domain to the Ingress Controller IP.
- Use A record for IP or CNAME for DNS.
- Example:
app.example.com → IP

7 TLS (HTTPS)



- Protect traffic with TLS.
- Use Kubernetes Secrets to store certificates.
- Ingress terminates TLS and forwards to Services.

```
kubectl create secret tls
```

8 REQUEST FLOW



- Client sends request.
- DNS resolves domain.
- Ingress receives request.
- Ingress routes to Service.
- Service forwards to Pods.
- Response flows back.

SAMPLE INGRESS YAML

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: default
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - app.example.com
    secretName: app-tls
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```

BROWSER TO INGRESS TO SERVICE TO POD ARCHITECTURE



BROWSER



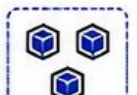
DNS



INGRESS
CONTROLLER



SERVICE



PODS

1 User enters
app.example.com
in the browser.

2 DNS resolves
the domain to
the Ingress
Controller IP.

3 Ingress Controller
receives the
request on
HTTP/HTTPS.

4 Ingress matches
the rule and
routes to the
correct Service.

5 Service load balances
the request to
healthy Pods.
Response returns
back to the user.

EXAMPLE ROUTING TABLE

HOST	PATH	SERVICE	PORT	DESCRIPTION
app.example.com	/	web-service	80	Serves the main website
app.example.com	/api	api-service	80	Routes API requests
admin.example.com	/	admin-service	80	Admin dashboard
shop.example.com	/	shop-service	80	E-commerce frontend



BEST PRACTICES

- ✓ Use Ingress for all external access.
- ✓ Use TLS for all public endpoints.
- ✓ Use host-based routing for separation.

- ✓ Use path-based routing for microservices.
- ✓ Keep Ingress rules simple and clear.
- ✓ Monitor Ingress Controller logs.

- ✓ Use meaningful hostnames.
- ✓ Test routing and failover.
- ✓ Secure your backend Services.

16. PROMETHEUS MONITORING LAB

COLLECT. QUERY. ALERT. INVESTIGATE.

OBJECTIVE

- Deploy and configure Prometheus
- Collect metrics from multiple sources
- Write PromQL queries
- Create alert rules
- Investigate and act on abnormal metrics

1 DEPLOY PROMETHEUS



- Deploy Prometheus in Kubernetes.
- Use a ConfigMap for prometheus.yml.
- Expose via Service (ClusterIP or NodePort).

```
helm install prometheus \
  prometheus-community/prometheus \
  -n monitoring --create-namespace
```

2 METRICS



- Metrics are time-series data.
- Format: `<metric_name>{<labels>} <value> <timestamp>`

Example:
`http_requests_total{code="200"} 1027`

3 TARGETS



- Targets are endpoints Prometheus scrapes.
- View and verify target health in the Targets page.
- Status: UP (green) or DOWN (red).

4 EXPORTERS



- Exporters expose metrics in Prometheus format.
- Common exporters:
 - node_exporter
 - cAdvisor
 - kube-state-metrics
 - blackbox_exporter

5 APPLICATION METRICS



- Instrument applications with client libraries.
- Expose /metrics endpoint.
- Monitor request rate, latency, errors, and business metrics.
- Example: Go, Python, Java, or Spring Boot apps.

6 NODE METRICS



- Collect CPU, memory, disk, network, and filesystem metrics.
- Use node_exporter on each node.
- Example metrics:
 - `node_cpu_seconds_total`
 - `node_memory_MemAvailable_bytes`

7 PROMQL BASICS



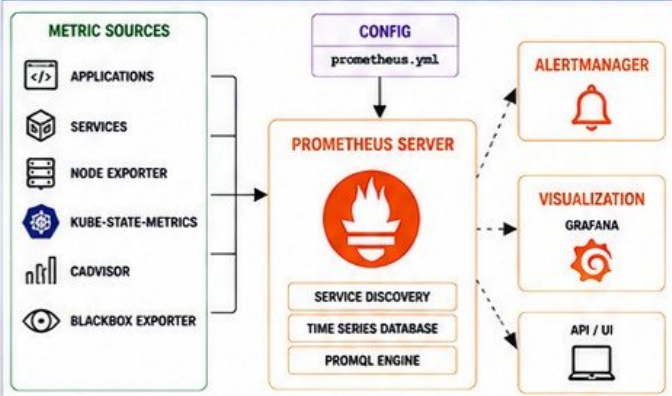
- Query the data powerfully with PromQL.
- Instant vector and range vector.
- Common functions:
 - `rate()`, `sum()`, `avg()`,
 - `irate()`, `topk()`, `by()`,
 - `increase()`

8 ALERT RULES



- Define alerting rules to detect issues early.
- Use Alertmanager to route notifications.
- Example: High CPU, High Memory, Down Targets, 5xx errors.

PROMETHEUS ARCHITECTURE



PROMQL EXAMPLES

```
rate(http_requests_total[5m]) # Request rate per second
sum(rate(http_requests_total[5m])) by (code) # Requests by status code
sum(node_cpu_seconds_total) by (instance) # CPU usage by instance
100 - (avg by (instance) (rate(node_cpu_seconds_total(mode="idle")[5m])) * 100) # CPU usage %
sum(node_memory_MemAvailable_bytes) by (instance) # Available memory
topk(5, sum(rate(http_requests_total[5m])) by (instance)) # Top 5 instances by traffic
increase(http_requests_total{code=~"5.."}[15m]) # 5xx errors increase
sum(rate(container_cpu_usage_seconds_total[5m])) by (pod) # CPU usage by pod
```

SIMULATE CPU AND MEMORY PRESSURE



CPU PRESSURE (Example)

```
stress --cpu 4 --timeout 300s
```

OR

```
kubectl run cpu-stress --image=polinux/stress \
  --restart=Never --rm -it -- \
  stress --cpu 4 --timeout 300s
```



MEMORY PRESSURE (Example)

```
stress --vm 2 --vm-bytes 1G --timeout 300s
```

OR

```
kubectl run mem-stress --image=polinux/stress \
  --restart=Never --rm -it -- \
  stress --vm 2 --vm-bytes 1G --timeout 300s
```

LAB EXERCISES

- 1 DEPLOY PROMETHEUS**
Install Prometheus and verify it is running.
- 2 ADD TARGETS**
Add node_exporter, cAdvisor, and a sample app.
- 3 VIEW METRICS**
Explore metrics in the Expression Browser.
- 4 PROMQL PRACTICE**
Write queries to answer common questions.
- 5 CREATE ALERT RULES**
Create rules for high CPU and memory usage.
- 6 SIMULATE CPU PRESSURE**
Use stress tool to generate CPU load.
- 7 SIMULATE MEMORY PRESSURE**
Use stress tool to consume memory.
- 8 INVESTIGATE ABNORMAL METRICS**
Identify spikes and understand the cause.
- 9 RECEIVE ALERTS**
Trigger alerts and verify notifications.
- 10 CLEAN UP**
Remove load and return system to normal.

ALERT RULE EXAMPLE (YAML)

```
groups:
- name: system-alerts
  rules:
  - alert: HighCPUUsage
    expr: 100 - (avg by (instance) (rate(node_cpu_seconds_total(mode="idle")[5m])) * 100) > 80
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "High CPU usage on {{ $labels.instance }}"
      description: "CPU usage is above 80% for more than 5 minutes."
```

INVESTIGATE ABNORMAL METRICS



DETECT
Alert fires or metric spikes



INVESTIGATE
Check dashboards, logs, and metrics



IDENTIFY ROOT CAUSE
Understand why the issue happened



RESOLVE
Fix the issue and validate recovery

USEFUL DASHBOARDS IN GRAFANA



NODE EXPORTER



KUBERNETES CLUSTER



POD OVERVIEW



CPU / MEMORY USAGE



APPLICATION METRICS



ALERTING DASHBOARD



PRO TIP

Good monitoring is not about more charts. It is about the right metrics, the right alerts, and fast root-cause analysis.

- ✓ Start simple.
- ✓ Monitor what matters.
- ✓ Trust but verify.
- ✓ Automate alerts.
- ✓ Learn from every incident.
- ✓ Keep improving your dashboards.

17. GRAFANA OBSERVABILITY LAB

VISUALIZE. ANALYZE. ACT.



OBJECTIVE

- Connect Grafana to Prometheus
- Build meaningful dashboards
- Visualize metrics and alerts
- Create an operations dashboard
- Gain real-time observability

1 CONNECT PROMETHEUS



- Add Prometheus as a data source.
- Test the connection.
- Set as default.

URL: <http://prometheus-server:9090>
Access: Server (default)
Type: Prometheus

2 BUILD DASHBOARDS



- Organize with folders.
- Add panels with PromQL queries.
- Use variables for flexibility.
- Save and share.

3 KEY METRICS TO VISUALIZE



- Infrastructure metrics
- Application health
- Kubernetes cluster metrics
- Alerts and events
- Performance and capacity

4 ALERT VISUALIZATION

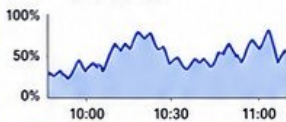


- View alert status in panels.
- See firing vs resolved.
- Drill down to root cause.
- Link to Alertmanager.



CPU

Monitor CPU usage and trends.

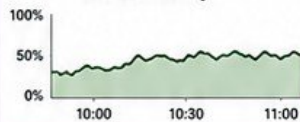


PromQL:
100 - (avg by(instance)(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)



MEMORY

Track memory usage and availability.

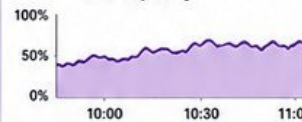


PromQL:
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100



DISK

Monitor disk usage and capacity.

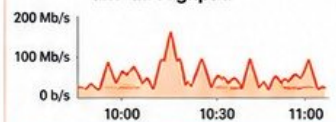


PromQL:
(1 - (node_filesystem_avail_bytes{fstype=~"tmpfs|overlay|squashfs"} / node_filesystem_size_bytes{fstype=~"tmpfs|overlay|squashfs"})) * 100



NETWORK

Track network I/O and throughput.

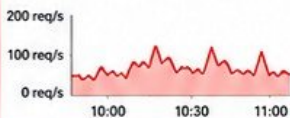


PromQL:
rate(node_network_receive_bytes_total[5m]) + rate(node_network_transmit_bytes_total[5m])



APPLICATION HEALTH

Monitor request rate, latency, errors, and availability.

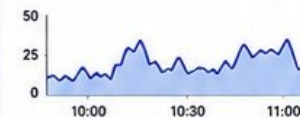


PromQL:
sum(rate(http_requests_total[5m])) by (job, status)



KUBERNETES METRICS

Visualize cluster, node, pod, and container metrics.



PromQL:
sum(kube_pod_status_phase{phase="Running"}) by (namespace)



ALERT VISUALIZATION

See active alerts and alert history.

ALERTS		
Firing	High CPU Usage	2m
Firing	High Memory Usage	1m
Resolved	Disk Space Low	15m

View in: Alert Manager



OPERATIONS DASHBOARD

Build a practical dashboard for daily operations.

- Overview panels
- Drill-down panels
- Alert summary
- Top talkers / heavy hitters
- Capacity and saturation

DASHBOARD BUILDING GUIDE



ADD PANEL

Choose visualization (Time series, Gauge, Stat, Bar, Table).



WRITE PROMQL

Test queries in Explore. Validate results.



CUSTOMIZE PANEL

Set title, unit, thresholds, legend, and colors.



ADD VARIABLES

Make dashboards dynamic (namespace, instance, job).



GROUP AND ORGANIZE

Use rows, collapsible panels, and folders.



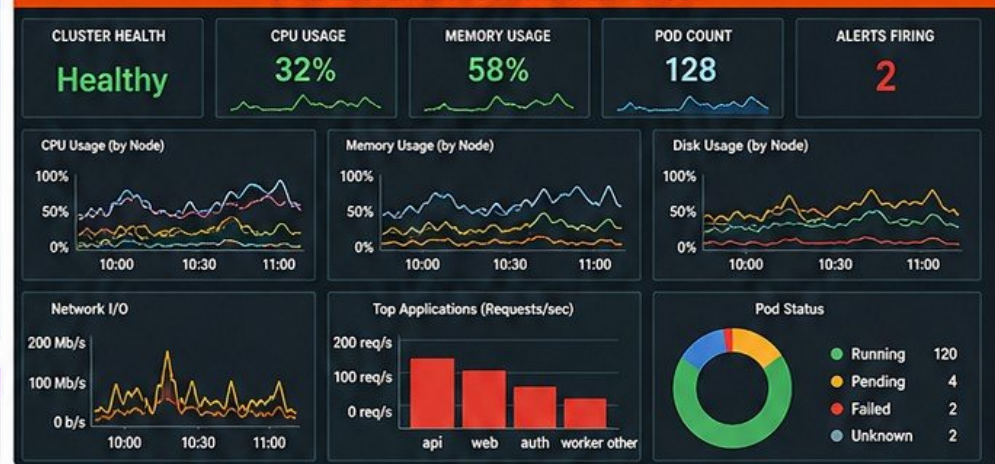
SAVE AND SHARE

Save dashboard. Share via folder or link.

PROMQL QUICK REFERENCE

rate(metric[5m])	Rate per second
sum(metric) by (label)	Sum by label
avg(metric) by (label)	Average by label
max(metric) by (label)	Max by label
topk(5, metric)	Top 5 series
increase(metric[1h])	Increase over time

SAMPLE OPERATIONS DASHBOARD LAYOUT



BEST PRACTICES

- ✓ Keep dashboards simple and clear.
- ✓ Use meaningful titles and units.
- ✓ Focus on SLIs and SLOs.
- ✓ Alert on symptoms, investigate root cause.
- ✓ Review and improve regularly.

18. CENTRALIZED LOGGING AND INCIDENT INVESTIGATION

SEE EVERYTHING. FIND THE TRUTH. RESOLVE FASTER.



OBJECTIVE

- Centralize logs from all sources
- Search, filter, and correlate logs
- Investigate incidents quickly
- Trace failures across components
- Improve MTTR with proper logging

1 APPLICATION LOGS



- Logs from your applications.
- Business events, errors, and performance.
- Use structured logging.

2 CONTAINER LOGS



- Logs from Docker containers.
- stdout / stderr streams.
- Collected by logging driver or agent.

3 KUBERNETES LOGS



- Pod logs.
- Events.
- Audit logs.
- Control plane component logs.

4 LINUX SYSTEM LOGS



- System events.
- Auth logs.
- Services.
- Kernel messages.
- Use journalctl and rsyslog.

5 STRUCTURED LOGGING



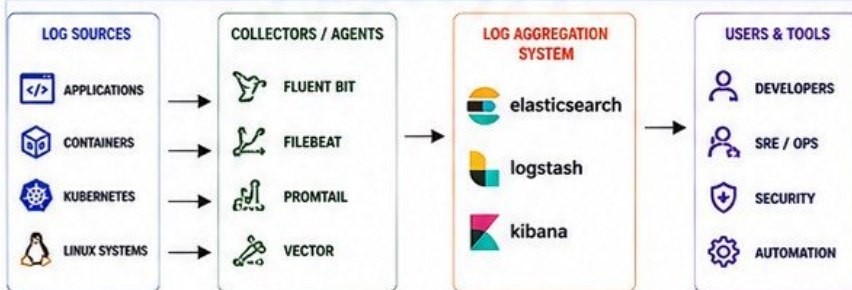
- Log in JSON format.
- Consistent fields.
- Easier parsing, search, and correlation.

6 LOG AGGREGATION



- Collect logs from all sources.
- Central storage.
- Index for fast search.

LOG AGGREGATION ARCHITECTURE



7 SEARCH AND FILTERING



- Full-text search across all logs.
- Filter by fields, time, level, service, pod, host.
- Use saved searches and views.

EXAMPLE FILTERS

```

level:ERROR
service:api-service
kubernetes.namespace:production
host:web-01
@timestamp:[now-30m TO now]
message:*timeout*
  
```

CORRELATING LOGS WITH METRICS

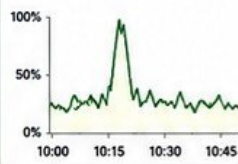
- Compare logs with metrics timelines.
- Validate spikes, errors, and latency.
- Use common labels: service, instance, pod, host.

LOG VIEW

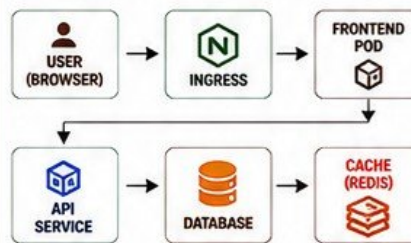
```

10:15:20 ERROR Payment failed
10:15:21 Timeout connecting to DB
10:15:22 Retrying transaction
10:15:30 ERROR Transaction failed
  
```

METRICS VIEW (CPU)



9 TRACE AN APPLICATION FAILURE ACROSS MULTIPLE COMPONENTS



Investigate step-by-step using logs from each component and correlate with metrics.

10 DEBUGGING INSIGHT

START WITH SYMPTOMS, THEN NARROW THE FAILURE DOMAIN

- 1 Observe the symptom.
- 2 Identify the affected user or request.
- 3 Check the most upstream component.
- 4 Move downstream one component at a time.
- 5 Correlate logs and metrics.
- 6 Find the first point of failure.
- 7 Validate root cause.
- 8 Fix and verify recovery.

COMMON LOG SOURCES AND LOCATIONS

SOURCE	WHAT IT CONTAINS	WHERE TO FIND
Application Logs	Business logs, errors, info	/var/log/app/ or stdout
Container Logs	Container stdout/stderr	/var/lib/docker/containers/*/*log
Kubernetes Pod Logs	Pod logs	kubectl logs <pod> -n <namespace>
Kubernetes Events	Cluster events	kubectl get events -A
Linux System Logs	System, auth, services	/var/log/syslog or journalctl
Kubernetes Audit Logs	API calls, auth, changes	/var/log/kubernetes/audit.log

USEFUL COMMANDS

kubectl logs <pod> -n <ns>	View pod logs
kubectl logs -f <pod> -n <ns>	Follow pod logs
kubectl get events -A --sort-by=.lastTimestamp	View events
journalctl -u <service> -f	Follow service logs
grep "ERROR" /var/log/syslog	Search system logs
curl -s http://localhost:9090/metrics	Check metrics endpoint

INCIDENT INVESTIGATION WORKFLOW



DETECT

Symptom or alert received



COLLECT

Gather logs, metrics, and events



CORRELATE

Match logs with metrics timeline



NARROW DOWN

Identify the failing component



FIND ROOT CAUSE

Determine why it happened



RESOLVE

Fix the issue



VERIFY

Confirm recovery and monitor



DOCUMENT

Write postmortem and lessons



BEST PRACTICES

- ✓ Log in structured JSON.
- ✓ Include timestamps and correlation IDs.
- ✓ Use consistent log levels.
- ✓ Centralize all logs.
- ✓ Keep logs with retention policy.
- ✓ Secure log data.
- ✓ Correlate logs with metrics and traces.
- ✓ Automate alerting and notifications.
- ✓ Practice incident investigations regularly.
- ✓ Learn from every incident.

19. END-TO-END DEVOPS HOME LAB

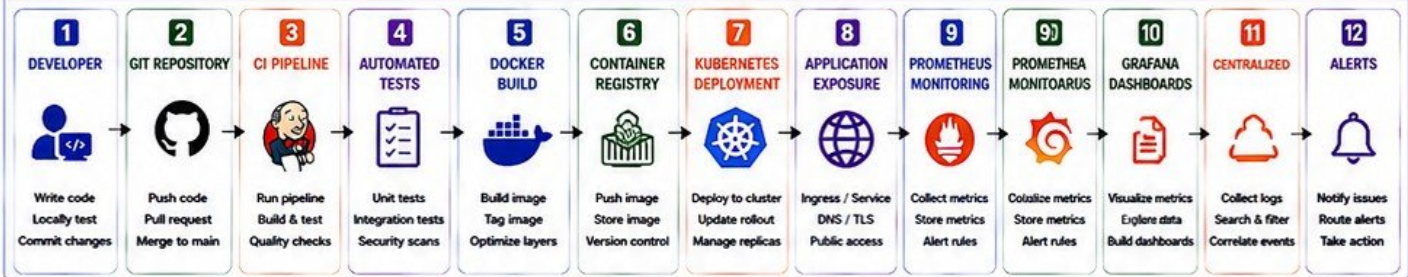
BUILD. DEPLOY. OBSERVE. FAIL. RECOVER. REPEAT.



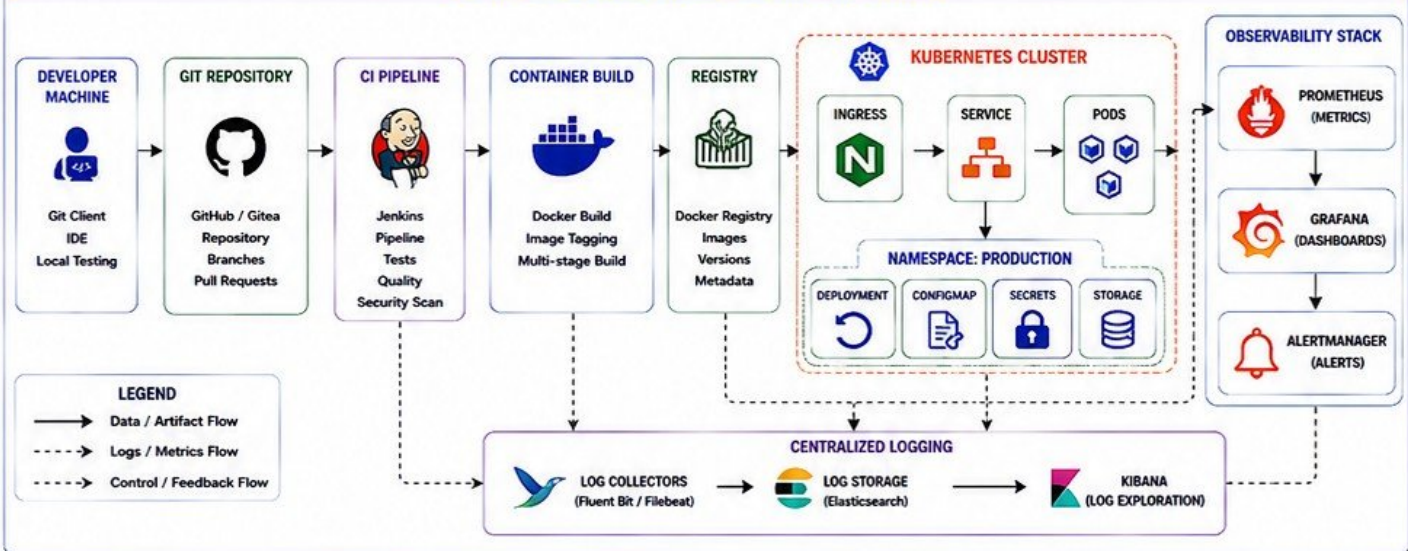
OBJECTIVE

- Build a complete DevOps workflow
- Verify deployments
- Simulate failures
- Rollback and recover
- Validate the recovery

COMPLETE DEVOPS WORKFLOW



END-TO-END DEVOPS HOME LAB ARCHITECTURE



✓ DEPLOYMENT VERIFICATION

- Verify rollout status
- Check pods are running
- Validate service endpoints
- Test application functionality
- Confirm metrics are collected
- Confirm logs are flowing

USEFUL COMMANDS

```
kubectl get pods -n prod
kubectl get svc -n prod
kubectl rollout status deploy/app -n prod
curl -I https://app.example.com/health
kubectl get ingress -n prod
```

⚠ FAILED DEPLOYMENT SIMULATION

- Push a broken image
- Introduce config error
- Break environment variable
- Simulate dependency failure
- Remove required secret
- Exhaust resource limits

EXAMPLE COMMAND (BREAK IT)

```
# Push a bad image tag
docker tag app:latest registry/app:bad
docker push registry/app:bad
# Deploy bad image
kubectl set image deploy/app app:registry/app:bad -n prod
```

↺ ROLLBACK

- Detect failed deployment
- Identify last working version
- Rollback to previous revision
- Verify application recovery
- Check metrics and logs

ROLLBACK COMMANDS

```
kubectl rollout history deploy/app -n prod
kubectl rollout undo deploy/app -n prod
kubectl rollout status deploy/app -n prod
```

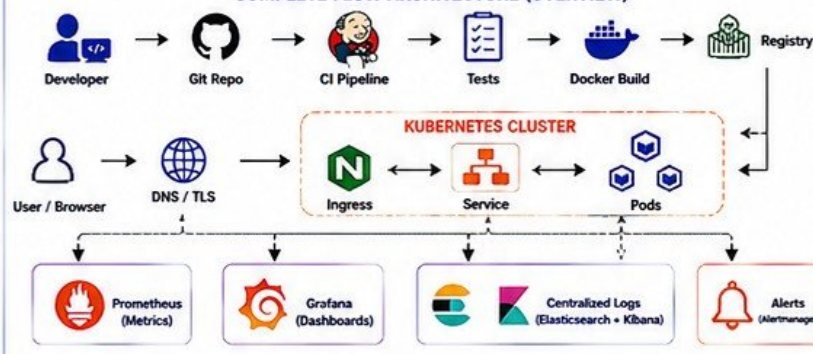
✓ RECOVERY VERIFICATION

- Confirm pods running and healthy
- Validate application endpoints
- Confirm metrics normalizing
- Confirm logs clean
- Confirm alerts resolved

VERIFY RECOVERY

```
kubectl get pods -n prod
kubectl get svc -n prod
curl -I https://app.example.com/health
kubectl get events --sort-by=.lastTimestamp
```

COMPLETE FLOW ARCHITECTURE (OVERVIEW)



KEY TAKEAWAYS

- ✓ You built a complete DevOps pipeline from code to production.
- ✓ You verified successful deployments.
- ✓ You simulated failures and observed their impact.
- ✓ You rolled back to a stable version.
- ✓ You recovered and verified the system.
- ✓ You have logs, metrics, and alerts for full visibility.
- ✓ This is how real-world DevOps teams operate.



PRODUCTION MINDSET

- Expect failures. Prepare for failures.
- Automate everything you can.
- Observe everything.
- Measure everything.
- Improve continuously.

20. PRODUCTION FAILURE CHALLENGE

BREAK THE HOME LAB INTENTIONALLY AND RECOVER IT.



OBJECTIVE

- Diagnose real-world failures
- Narrow down the root cause
- Fix the issue and restore service
- Improve systems and prevent recurrence
- Build production-grade troubleshooting skills

FAILURE SCENARIOS

1 CI PIPELINE FAILS



- Build fails
- Tests fail
- Dependency error
- Artifact not generated

Focus: Pipeline logs, test reports, config

2 DOCKER IMAGE IS BROKEN



- Build succeeds but image is invalid
- Missing files
- Wrong base image
- Vulnerability blocks run

Focus: Dockerfile, build logs, image inspection

3 KUBERNETES POD CRASHES



- CrashLoopBackOff
- OOMKilled
- Liveness probe fails
- Application error

Focus: Pod logs, describe, events, resource limits

4 SERVICE CANNOT REACH ANOTHER SERVICE



- DNS failure
- Network policy blocks
- Wrong Service name
- Port mismatch

Focus: DNS, network policies, services, endpoints

5 APPLICATION LOSES DATABASE CONNECTIVITY



- DB down
- Wrong credentials
- Connection pool exhausted
- Network policy / firewall

Focus: DB logs, secrets, network, app config

6 NODE REACHES RESOURCE LIMITS



- High CPU usage
- High memory usage
- Disk full
- PID / inode exhaustion

Focus: Node metrics, top, df -h, dmesg

7 MONITORING TARGET DISAPPEARS



- Prometheus target down
- Exporter not running
- Service/endpoint changed
- Network issue

Focus: Prometheus targets, exporter logs, service

8 DEPLOYMENT INTRODUCES ERRORS



- New version has bugs
- Config errors
- Breaking changes
- Failing migrations

Focus: Rollout history, logs, events, metrics

9 CONFIGURATION DRIFT OCCURS



- Manual changes
- Config out of sync
- Wrong replicas / settings
- Inconsistent environments

Focus: Git vs cluster state, kubectl diff, IaC

10 SECRET OR CONFIGURATION IS INCORRECT



- Wrong secret value
- Missing secret
- Expired credentials
- Bad configuration

Focus: Secrets, configmaps, environment variables

TROUBLESHOOTING APPROACH (USE EVERY TIME)



1 IDENTIFY THE SYMPTOM

What is failing?
What is the impact?



2 DEFINE THE SCOPE

Which component is affected?
Is it isolated?



3 GATHER EVIDENCE

Logs, metrics, events, configs, history



4 FIND THE ROOT CAUSE

Why did this happen?
Be specific.



5 FIX THE ISSUE

Apply the right change or rollback.



6 VERIFY RECOVERY

Confirm the fix.
Monitor closely.



7 PREVENT RECURRENCE

Document lesson.
Improve monitoring, alerts, and tests.

SAMPLE END-TO-END FAILURE & RECOVERY LAB



CODE PUSH

Failure
Bad code breaks build

Recover
Fix code
Push again



CI PIPELINE

Failure
Tests fail
Pipeline stops

Recover
Fix tests / config
Rerun pipeline



BUILD IMAGE

Failure
Dockerfile error
Image build fails

Recover
Fix Dockerfile
Build again



PUSH IMAGE

Failure
Registry auth push fails

Recover
Fix credentials
Push again



K8S DEPLOY

Failure
Pod crash / Deployment fails

Recover
Fix issue
Rollback if needed
Redeploy



APP SERVED

Failure
503 / 5xx
User impact

Recover
Verify service
Health restored

USEFUL COMMANDS

<code>kubectl get pods -A</code>	List all pods
<code>kubectl describe pod <pod></code>	Inspect pod details
<code>kubectl logs <pod> -f</code>	Stream pod logs
<code>kubectl get events -A</code>	View events
<code>kubectl top nodes</code>	Check node resources
<code>kubectl top pods -A</code>	Check pod resources
<code>kubectl rollout history deploy/<app></code>	View rollout history
<code>kubectl rollout undo deploy/<app></code>	Rollback deployment
<code>kubectl get svc,pods,endpoints -A</code>	Service connectivity
<code>kubectl get configmap,secret -A</code>	Check configs/secrets
<code>kubectl diff -f <file.yaml></code>	Check drift (IaC)

SUCCESS CRITERIA

- ✓ You can identify the problem quickly.
- ✓ You find the root cause.
- ✓ You recover the system completely.
- ✓ You verify the fix with metrics and logs.
- ✓ You document what happened.
- ✓ You prevent it from happening again.

DEBUGGING INSIGHT



START WITH SYMPTOMS,
THEN NARROW THE FAILURE DOMAIN.

Observe → Isolate → Analyze → Fix → Verify → Improve